MANUAL MAESTRO DE DESARROLLO (STACK NEXT.JS | TS | SHADCN)

CAPÍTULO 1: FILOSOFÍA Y SETUP

1.1 Filosofía Arquitectónica: Por Qué Este Stack

La elección del stack (Next.js App Router, TypeScript, Tailwind CSS y ShadCN/UI) no es accidental; es una **decisión estructural** diseñada para resolver los desafíos de escalabilidad, rendimiento y mantenibilidad inherentes a las Single Page Applications (SPAs) tradicionales.

1. **Next.js 15 (App Router)**: Representa un cambio de paradigma que disuelve la dicotomía tradicional entre *frontend* y *backend*. Su principal ventaja es el uso de **React Server Components (RSC)**. Al migrar la lógica de negocio, el formateo de datos y el acceso a la base de datos al entorno seguro del servidor, se reduce drásticamente el peso del *bundle* de JavaScript enviado al cliente y se mejora el **Time to Interactive (TTI)**.
2. **TypeScript**: Es un **contrato vinculante** que asegura la integridad de los datos a través de toda la aplicación. La seguridad de tipos debe ser prioritaria, permitiendo que el compilador actúe como la primera línea de defensa contra errores lógicos.
3. **ShadCN/UI y Tailwind CSS**: Abandonamos las bibliotecas de componentes monolíticas opacas (como Material UI) en favor de un sistema de diseño basado en la **"Propiedad del Código"**. ShadCN utiliza primitivas *Headless* (Radix UI) para la lógica funcional y accesibilidad, y Tailwind para los estilos visuales. Esto nos da **control total sobre el diseño**, eliminando la "deuda de abstracción" y permitiendo modificaciones directas al código fuente de los componentes.

1.2 Instalación y Configuración Inicial

La base del proyecto se establece mediante la inicialización de Next.js y la adición del sistema de diseño ShadCN/UI.

1. Inicialización de Next.js (App Router)

Se recomienda encarecidamente inicializar el proyecto con la opción de TypeScript y el `src` directory para aislar el código fuente:

```
npm create next-app@latest
```

2. Inicialización de ShadCN/UI

Una vez configurado Tailwind CSS, se utiliza el instalador de ShadCN para configurar el archivo `components.json` y copiar los *helpers* de utilidad esenciales:

```
npx shadcn-ui@latest init
```

Durante la configuración de ShadCN, se debe **mantener `cssVariables: true`** en `components.json` para habilitar la arquitectura de tokens semánticos (Ver Capítulo 3).

CAPÍTULO 2: ARQUITECTURA NEXT.JS (APP ROUTER)

2.1 Server Components vs. Client Components

En la arquitectura del App Router, la **frontera de red** es una decisión de diseño explícita. Los React Server Components (RSC) y los Client Components (CC) no son conceptos nuevos de renderizado (como SSR vs. CSR), sino **unidades de computación** que definen dónde se ejecuta el código.

Dimensión Arquitectónica	Server Components (RSC)	Client Components (CC)

Ubicación de Ejecución	Servidor (Exclusivo)	Cliente (Navegador)
Acceso a Recursos	Base de datos, sistema de archivos, secretos	`window`, `localStorage`, Hooks de estado (`useState`, `useEffect`)
Directiva	Por defecto (No se necesita directiva)	`'use client'` (Opt-in obligatorio)
Bundle de JS	Su código **nunca** se envía al cliente.	Requieren hidratación y contribuyen al tamaño del *bundle*.
Flujo de Datos	Pueden `await` llamadas asíncronas de datos (Fetch at the Source).	Reciben datos serializables como *props* desde un RSC padre.

Cuándo Usar Cada Uno

- **RSC (Por Defecto)**: Debe ser el estado predeterminado para toda la aplicación. Úselo para:
  - **Data Fetching**: Consulta a la base de datos o APIs externas.
  - **Lógica de Negocio**: Formateo de datos o validaciones complejas.
  - **Estructura Estática**: *Layouts*, *Footers*, contenido editorial.
- **CC (Opt-in)**: Úselo **solo** cuando se requiera interactividad del navegador:
  - Manejo de estados locales (`useState`, `useReducer`).
  - *Event listeners* (`onClick`, `onChange`).
  - Librerías que dependen del ciclo de vida de React (ej. TanStack Query).

Anti-Patrón a Evitar: Envolver una página completa (`page.tsx`) con `'use client'`. Esto convierte la ruta en una SPA tradicional, perdiendo los beneficios de RSC. Adopte el **Principio de "Hojas Interactivas"**: empuje la frontera de red lo más abajo posible, importando pequeños componentes de cliente dentro de grandes árboles de servidor estáticos.

2.2 Estructura de Carpetas y Modularización

Se recomienda el uso del directorio `/src` para aislar el código fuente y una **arquitectura basada en características (Feature-Driven)** sobre una estructura horizontal.

Directorio	Propósito Arquitectónico	Justificación
`/src/app`	**Capa de Enrutamiento (Orquestación)**	Define URLs, metadatos, y compone *Layouts*. Debe ser una capa fina.

`/src/features`	**Núcleo de la Lógica de Negocio (Domain Modules)**	Cohesión: Todo lo relacionado con un dominio (`/auth`, `/projects`) reside aquí, incluyendo componentes, *actions*, *queries* y *schemas* de Zod.
`/src/components/ui`	**UI Kit Corporativo**	Componentes base, agnósticos al negocio (Botones, Modales, Tablas). Aquí residen los archivos de ShadCN/UI copiados.
`/src/lib`	**Infraestructura y Adaptadores**	Configuración de ORM (`db.ts`), clientes de pago (`stripe.ts`), *helpers* genéricos (`utils.ts`, `cn()`).
`/src/hooks`	**Hooks Globales Reutilizables**	Lógica de estado compartida que no está ligada a un *feature* específico.

2.3 Persistencia de Estado: Layouts y Templates

La gestión del estado persistente en la navegación se define mediante la elección entre `layout.tsx` y `template.tsx`.

• **Layouts (`layout.tsx`)**: No se desmontan al navegar entre rutas hijas que comparten el mismo *layout*. **Preservan la instancia** del componente y, por lo tanto, el estado interactivo (ej. el texto en un input de búsqueda global o la posición de scroll). Son la base para emular la sensación fluida de una SPA.

• **Templates (`template.tsx`)**: Crean una **nueva instancia (montaje/desmontaje)** en cada navegación, incluso entre rutas hermanas. Son útiles para casos donde se requiere **resetear el estado** (ej. un formulario) o registrar un evento de analíticas (`Page View`) en cada entrada a la ruta.

Para la escalabilidad, se recomienda el patrón de **Layouts Anidados**, donde las capas de la aplicación (Root, Dashboard, Configuración) se definen jerárquicamente, asegurando que al navegar, solo se re-renderice el segmento de contenido más profundo (`children`).

CAPÍTULO 3: SISTEMA DE DISEÑO (SHADCN + TAILWIND)

3.1 Implementación de Componentes ShadCN: La Propiedad del Código

ShadCN/UI se basa en una filosofía de **"Copiar, no instalar"**. Los componentes se escafoldean directamente en `src/components/ui`.

• **Fundación *Headless***: Los componentes utilizan primitivas funcionales robustas de **Radix UI** para gestionar la accesibilidad (foco, teclado, ARIA).

• **Composición**: Los componentes complejos se construyen mediante **Componentes Compuestos**, lo que permite una flexibilidad declarativa superior. Por ejemplo, el componente `Card` se expone a través de sus partes modulares (`CardHeader`, `CardTitle`, `CardContent`,

`CardFooter`), permitiendo al desarrollador ensamblar la UI sin necesidad de "hackear" las *props* del componente raíz.

```
<Card className="w-[350px]">
  <CardHeader>
    <CardTitle>Crear Proyecto</CardTitle>
    <CardDescription>Despliega tu nuevo sitio en un clic.</CardDescription>
  </CardHeader>
  {/* ... contenido ... */}
  <CardFooter>
    {/* ... botones ... */}
  </CardFooter>
</Card>
```

3.2 La Utilidad cn(): Fusión Determinista de Clases

En Tailwind CSS, el estilado se realiza mediante la gestión de cadenas de texto. La función `cn()` es un requisito arquitectónico que garantiza la resolución determinista de conflictos CSS, combinando **`clsx`** y **`tailwind-merge`**.

1. **`clsx`**: Maneja la construcción condicional de clases, compactando *arrays*, objetos y expresiones booleanas en una cadena única.
2. **`tailwind-merge` (`twMerge`)**: Resuelve los conflictos de utilidad de Tailwind. Si se aplican clases mutuamente excluyentes (ej. `p-4` y `p-8`), `twMerge` analiza semánticamente las clases y asegura que la última definida en la cadena **gane**, eliminando la clase anterior.

Uso Obligatorio: Todo componente reutilizable debe usar `cn()` para fusionar sus clases base inmutables con la `className` inyectada por el consumidor.

```
// Implementación robusta en un componente Card [32]
className={cn(
  // Clases Base: Identidad visual inmutable
  "rounded-xl border bg-card text-card-foreground shadow",
  // Clases Inyectadas: Permiten la sobreescritura del consumidor
  className
)}
```

3.3 Buenas Prácticas de Estilo: Tokens y Minimalismo

1. Prohibición de los "Valores Mágicos"

Se prohíbe el uso de la sintaxis de corchetes de Tailwind para inyectar valores arbitrarios que no existen en el sistema de diseño (ej. `w-[357px]` o `mt-[13px]`). Esto rompe la consistencia visual. Se debe adherir estrictamente a la escala de espaciado predeterminada de Tailwind o extender el tema de forma controlada en `tailwind.config.ts`.

2. Estrategia de Color Semántico

Para permitir la tematización dinámica (Modo Oscuro, *Branding*), se utiliza una capa de abstracción que mapea **Tokens Semánticos** a Variables CSS.

- **Nivel Semántico**: Nombres funcionales como `primary`, `destructive`, `muted`.
- **Nivel Primitivo**: Los valores reales se definen en variables HSL en `app/globals.css` dentro de `:root`.

Para cada color de superficie, existe un par de **`background`** y **`foreground`** garantizado para ser accesible.

```
// Uso en el componente: agnóstico al valor real del color
<div className="bg-primary text-primary-foreground hover:bg-primary/90">
```

```
  Confirmar Transacción
</div>
```

3. Estética de "Lujo Silencioso"

Para lograr una estética profesional y minimalista (tipo Linear/Vercel), se recomienda usar una paleta neutra controlada (ej. **Stone** o **Slate** en la configuración de ShadCN), que define la "temperatura" de la interfaz. Los tokens esenciales (`background`, `foreground`, `border`, `input`) deben usarse rigurosamente en lugar de colores literales de la paleta primitiva.

CAPÍTULO 4: GESTIÓN DE DATOS Y ESTADO

4.1 La Guía Definitiva para Formularios: RHF + Zod + Server Actions

La gestión de formularios se basa en una **"trinidad arquitectónica"** que garantiza rendimiento, integridad de datos y seguridad.

1. React Hook Form (RHF) y Rendimiento

Se prioriza la arquitectura de **componentes no controlados** de RHF. En lugar de re-renderizar el formulario completo con cada pulsación de tecla (como lo haría un input controlado con `useState`), RHF registra referencias (*refs*) al DOM. Esto resulta en un **aislamiento de renderizado** que optimiza el rendimiento.

2. Validación Isomórfica con Zod

Zod es el estándar para la definición de esquemas, ya que es "TypeScript-first" e infiere los tipos estáticos directamente.

• **Beneficio**: El mismo esquema Zod define las reglas de validación en *runtime* (cliente) y los tipos en *compile-time* (servidor). Esto garantiza la **consistencia isomórfica**.

• **Integración**: Se utiliza `@hookform/resolvers/zod` para transformar la salida de Zod al formato de errores que RHF espera.

3. Integración con Server Actions

Las **Server Actions** son la primitiva estándar para mutaciones de datos en Next.js, eliminando la necesidad de endpoints API REST manuales.

• **Seguridad**: Es innegociable validar la entrada en el servidor. La Server Action debe usar `schema.safeParse` para validar los datos recibidos (FormData).

• **Hidratación de Errores**: Si la Server Action falla (ej. error de base de datos o validación de negocio), retorna un objeto de estado con los errores serializados. En el cliente, el *hook* **`useActionState`** (o `useFormState`) observa este estado y utiliza `setError` de RHF para inyectar los errores del servidor en los campos del formulario, ofreciendo una UX fluida.

```
// Server Action (actions.ts - Lógica de Validación y Error) [53]
'use server'
import { z } from 'zod';

const createProjectSchema = z.object({ /* ... */ });

export async function createProject(prevState: ActionState, formData: FormData) {
  // 1. Validación de Datos (Zod)
  const validatedFields = createProjectSchema.safeParse(/* ... */);

  if (!validatedFields.success) {
    return { errors: validatedFields.error.flatten().fieldErrors } // Retorna errores serializados
  }
  // ... Mutación DB ...
}
```

4.2 Tablas Complejas: TanStack Table y Debounce

TanStack Table v8 es una librería **"headless"**; solo gestiona la lógica de la tabla (ordenamiento, filtrado, paginación) y expone una API.

• **Tipado Riguroso**: Para asegurar la seguridad de tipos (`TData`), se recomienda utilizar **`createColumnHelper`** en lugar de *Accessor Keys* simples para definir las columnas.

• **Filtrado de UX**: Filtrar en cada pulsación de tecla causa re-renderizados costosos. Para mitigar esto, se debe implementar un componente **`DebouncedInput`** que mantenga un estado local y solo propague el cambio al estado de la tabla (o URL) después de un periodo de inactividad (ej. 300-500ms), previniendo el "bombardeo" de peticiones.

• **Paginación Servidor**: Si se utiliza paginación manual (del lado del servidor), es vital pasar la propiedad `pageCount` (calculada por el *backend*) a la instancia de `useReactTable` para que los controles de navegación se rendericen correctamente.

4.3 Gestión de Estado en la URL: La Fuente de la Verdad

El principio rector es: **Si el estado afecta lo que el usuario ve y debe ser compartible, pertenece a la URL**. Esto incluye filtros de búsqueda, paginación y ordenamiento.

• **Limitaciones Nativas**: El *hook* nativo `useSearchParams` de Next.js presenta fricciones: todo es *string* (requiere *casteo* manual) y las actualizaciones son imperativas (código verboso).

• **Solución Type-Safe: nuqs**: La librería **nuqs** (Next.js URL Query State) proporciona una API similar a `useState` pero que persiste en la URL con **seguridad de tipos completa**. Utiliza *parsers* (ej. `parseAsInteger`, `parseAsString`) para definir el tipo de dato esperado, eliminando el *casteo* manual.

```
// nuqs: Estado sincronizado y type-safe [63]
import { useQueryState, parseAsInteger } from 'nuqs';

const [page, setPage] = useQueryState(
  'page',
  parseAsInteger.withDefault(1) // Si ?page= no existe o es inválido, devuelve 1
);
// setPage(2) actualiza la URL a ?page=2 automáticamente
```

CAPÍTULO 5: ESTÁNDARES DE TYPESCRIPT

5.1 Reglas de Oro para el Tipado

El código limpio en React requiere disciplina en el tipado.

1. **Prohibición de `any`**: El uso de **`any`** está estrictamente prohibido. Si el tipo es desconocido, utilice **`unknown`** y obligue a la comprobación de tipos (*type narrowing*) explícita en tiempo de ejecución para mayor seguridad.
2. **`interface` vs. `type`**:
   - **Interfaces**: Utilícelas para definir **APIs públicas** y estructuras de objetos que necesitan ser **extendidas** o fusionadas (como las que se usarían para un sistema de diseño o librería).
   - **Type Alias**: Son la opción preferida para **props internos**, estados, **tipos de unión** (`type Status = 'success' | 'error'`), tuplas, y alias de primitivos. En el contexto de aplicaciones, la balanza se inclina hacia `type` para evitar la fusión accidental.
3. **Tipado de `children`**: Utilice **`React.ReactNode`**. Este es el tipo más robusto y permisivo, representando cualquier cosa que React pueda renderizar (elementos, strings, null, arrays).
4. **Evitar `React.FC`**: Se desaconseja explícitamente el uso de `React.FC` ya que añade implícitamente `children` incluso a componentes que no deberían aceptarlos. Se debe tipar `children` de forma explícita en las *props* si es necesario.

```
// Patrón recomendado para un componente [72]
type CardProps = {
  title: string;
  children: React.ReactNode; // Definición explícita
};

export const Card = ({ title, children }: CardProps) => { /* ... */ };
```

5.2 Tipado de Props y Respuestas de API

1. Tipado Estricto de Eventos

Al manejar eventos del DOM, debe especificarse el genérico del elemento HTML que dispara el evento, eliminando la necesidad de *casting*.

```
// Tipado preciso en la firma de la función [74]
const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
  // Acceso seguro a event.currentTarget.value
  setTerm(event.currentTarget.value);
};
```

2. Wrappers Genéricos para Respuesta API

Para lograr una seguridad de tipos de extremo a extremo, se deben utilizar *wrappers* genéricos para definir la forma de la respuesta de la API (el "sobre" de datos: `{ data: T, status: number }`).

```
// Definición del sobre de respuesta de la API [75]
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}

// La función genérica infiere el tipo de la carga útil automáticamente
async function request<T>(url: string): Promise<T> {
  const response = await fetch(url);
  const result: ApiResponse<T> = await response.json();
  return result.data; // Retorna solo T, el tipo de dato esperado
}
```

3. Uniones Discriminadas (Discriminadas Unions) para Estados

Este patrón es crucial para gestionar estados de UI complejos (`Cargando`, `Éxito`, `Error`), eliminando **combinaciones de estado inválidas** que el uso de booleanos separados permitiría.

```
// Tipado seguro para la gestión de estados [77]
type State =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: User } // 'data' solo existe si el status es 'success'
  | { status: 'error'; error: Error };

function Componente({ state }: { state: State }) {
  if (state.status === 'success') {
    // TypeScript estrecha el tipo, permitiendo acceso seguro a state.data
```

```
  console.log(state.data.name);
}
}
```

Analogía Final: Piense en este stack como la construcción de un rascacielos. **Next.js/RSC** es la cimentación y la estructura principal (eficiente y rápida de levantar, controlando dónde se coloca cada piso). **TypeScript** es el plano arquitectónico detallado que garantiza que cada viga esté en su lugar y sea del tipo correcto. **ShadCN/Tailwind** son los materiales de construcción estéticos, elegidos por su flexibilidad y la capacidad de modificarlos en el sitio (propiedad del código). Finalmente, **RHF/Zod/nuqs** son los sistemas internos (electricidad, plomería, HVAC) que aseguran que el flujo de datos dentro y fuera del edificio sea eficiente, seguro y rastreable.